

 ASCENT FORMATION

TP Rust — Analyseur de logs

Développement complet d'un outil en ligne de commande · 4 h

Formation « Langage Rust — débutant » · Marc DESCHAMPS
Version 2026

Sommaire

Présentation

- Contexte
- Objectifs
- Organisation
- Fichiers fournis

Le format des logs

Palier 0 — Le projet (15 min)

Palier 1 — Le niveau de gravité (30 min)

Palier 2 — Analyse d'une ligne (60 min)

- 2.1 L'erreur personnalisée ([erreur.rs](#))
- 2.2 L'entrée de log ([entree.rs](#))
- 2.3 La fonction d'analyse

Palier 3 — Chargement d'un journal (25 min)

Palier 4 — Statistiques, filtres et rapport (50 min)

Palier 5 — Le menu interactif (45 min)

- Comportement attendu
- Conseils

Bonus

Annexe — Aide-mémoire

Présentation

Contexte

Une équipe d'exploitation reçoit chaque jour les journaux (*logs*) de ses serveurs d'application. Pour diagnostiquer rapidement un incident, elle a besoin d'un outil en ligne de commande qui lit un journal, ignore les lignes mal formées, et permet, depuis un **menu**, d'afficher des statistiques, de filtrer les entrées et de produire un rapport.

Vous allez développer cet outil **de zéro**, en Rust.

Objectifs

À la fin du TP, vous aurez mobilisé l'ensemble des notions des jours 1 à 3 :

- créer et organiser un projet Cargo en modules (bibliothèque + programme) ;
- modéliser des données avec `struct` et `enum`, et leur associer des méthodes (`impl`) ;
- analyser du texte et convertir des valeurs ;
- gérer les cas d'échec avec `Option`, `Result`, l'opérateur `?` et une erreur personnalisée ;
- lire et écrire des fichiers, lire les saisies au clavier, exploiter les arguments ;
- respecter un contrat vérifié par des tests automatiques.

Organisation

Palier	Contenu	Durée indicative
0	Création du projet et des modules	15 min
1	Le niveau de gravité (<code>enum Niveau</code>)	30 min
2	Analyse d'une ligne (<code>EntreeLog</code> , <code>ErreurLog</code>)	60 min
3	Chargement d'un fichier	25 min
4	Statistiques, filtres et rapport	50 min
5	Le menu interactif	45 min
Bonus	Pour les plus rapides	—

Les paliers s'enchaînent : chacun s'appuie sur le précédent. **Validez chaque palier avec ses tests avant de passer au suivant.**

Fichiers fournis

Le dossier `fichiers_fournis/` contient :

- `logs/serveur.log` : un journal de 33 lignes, avec quelques pièges ;
- `tests/palier1_niveau.rs` à `tests/palier4_analyse.rs` : les tests de validation de chaque palier.

Ne modifiez pas les tests. Ils fixent le nom des types et des fonctions, leurs paramètres et leur comportement : votre code doit s'y conformer exactement.

Le format des logs

Chaque ligne du journal a la forme suivante :

```
2026-10-06 08:16:40 [ERROR] auth: compte bob bloqué après 3 échecs
└─ date ─┐ └─ heure ─┐ └─ niveau ─┐ └─ mod ─┐ └─ message ─┐
```

Une ligne est **valide** si, après suppression des espaces au début et à la fin :

1. elle contient au moins 4 morceaux séparés par des espaces : date, heure, niveau entre crochets, reste ;
2. la date a exactement la forme `AAAA-MM-JJ` (chiffres et tirets) ;
3. l'heure a exactement la forme `HH:MM:SS` (chiffres et deux-points) ;
4. le niveau est entre crochets et vaut `DEBUG`, `INFO`, `WARN` ou `ERROR`, sans tenir compte des majuscules ;
5. le reste contient `" : "` (deux-points puis espace) : ce qui précède est le **module**, ce qui suit est le **message** ;
6. le module n'est pas vide et ne contient pas d'espace ; le message n'est pas vide (espaces autour retirés).

Le message peut lui-même contenir `" : "` : seule la **première** occurrence sépare le module du message.

On ne vérifie pas que la date existe réellement (un 31 février est accepté) : seule la forme compte.

Dans un fichier, les **lignes vides** et les lignes commençant par `#` (commentaires) sont ignorées. Les autres lignes invalides ne doivent **pas** arrêter le chargement : elles sont mises de côté et signalées.

Palier 0 — Le projet (15 min)

1. Créez le projet :

```
cargo new analyseur_logs
cd analyseur_logs
```

2. Copiez dans ce dossier les dossiers `logs/` et `tests/` fournis.
3. Organisez le code en **bibliothèque + programme**. Le programme (`main.rs`) ne contient que l'interface utilisateur ; toute la logique est dans la bibliothèque, ce qui la rend testable. Créez les fichiers suivants dans `src/` :

Fichier	Contenu
<code>lib.rs</code>	Déclaration des modules
<code>niveau.rs</code>	<code>enum Niveau</code> (palier 1)
<code>erreur.rs</code>	<code>enum ErreurLog</code> (palier 2)
<code>entree.rs</code>	<code>struct EntreeLog</code> , <code>analyser_ligne</code> (palier 2)
<code>chargement.rs</code>	<code>charger_texte</code> , <code>charger_fichier</code> (palier 3)
<code>analyse.rs</code>	Statistiques, filtres, rapport (palier 4)
<code>main.rs</code>	Menu interactif (palier 5)

4. Dans `lib.rs`, déclarez les modules **publics** et réexportez leur contenu, pour que les tests puissent écrire simplement `use analyseur_logs::*` :

```
pub mod analyse;
pub mod chargement;
pub mod entree;
pub mod erreur;
pub mod niveau;

pub use analyse::*;
pub use chargement::*;
pub use entree::*;
pub use erreur::*;
pub use niveau::*;
```

Au début, les fichiers des paliers suivants peuvent rester vides. Pour qu'un module en utilise un autre : `use crate::niveau::Niveau` ;

Test du palier : `cargo build` compile sans erreur.

Lancer les tests d'un seul palier : `cargo test --test palier1_niveau`. La commande `cargo test` seule compile **tous** les fichiers de tests : elle échouera tant que tous les paliers ne sont pas écrits.

Palier 1 — Le niveau de gravité (30 min)

Dans `niveau.rs`, créez l'énumération `Niveau` avec quatre variantes, **dans cet ordre** : `Debug`, `Info`, `Warn`, `Error`.

```
#[derive(Debug, Clone, Copy, PartialEq, PartialOrd)]
pub enum Niveau { /* ... */ }
```

Grâce à `PartialOrd`, les variantes se comparent dans leur ordre de déclaration : `Niveau::Warn < Niveau::Error`. Cela servira au filtrage par niveau minimum.

Implémentez :

Élément	Comportement
<code>pub const TOUS: [Niveau; 4]</code>	Les quatre niveaux, du moins grave au plus grave (constante associée, dans le bloc <code>impl</code>)
<code>pub fn depuis_texte(texte: &str) -> Option<Niveau></code>	"warn", " WARN ", "Warn" → <code>Some(Niveau::Warn)</code> ; tout texte inconnu → <code>None</code>
<code>pub fn libelle(&self) -> &'static str</code>	"DEBUG", "INFO", "WARN", "ERROR"
<code>pub fn indice(&self) -> usize</code>	0 pour Debug, 1, 2, 3 pour Error
<code>impl fmt::Display for Niveau</code>	Affiche le libellé : <code>format!("{}", Niveau::Info)</code> donne <code>INFO</code>

Indices : `texte.trim().to_uppercase()` donne une `String` ; pour la comparer dans un `match` à des littéraux, utilisez `.as_str()`.

Test : `cargo test --test palier1_niveau` (6 tests).

Palier 2 — Analyse d'une ligne (60 min)

C'est le cœur du TP. Prenez le temps de bien découper le problème.

2.1 L'erreur personnalisée (`erreur.rs`)

```
#[derive(Debug, PartialEq)]
pub enum ErreurLog {
    FormatInvalide(String), // la ligne fautive, sans espaces autour
    Fichier(String),        // le chemin du fichier illisible
}
```

Implémentez `Display` :

- `FormatInvalide("xyz")` → ligne invalide : xyz
- `Fichier("a.log")` → impossible de lire le fichier a.log

2.2 L'entrée de log (`entree.rs`)

```
#[derive(Debug, Clone, PartialEq)]
pub struct EntreeLog {
    pub date: String,
    pub heure: String,
    pub niveau: Niveau,
    pub module: String,
    pub message: String,
}
```

Implémentez `Display` pour retrouver le format d'origine : `2026-10-06 08:00:01 [INFO] serveur: démarrage`.

2.3 La fonction d'analyse

```
pub fn analyser_ligne(ligne: &str) -> Result<EntreeLog, ErreurLog>
```

Elle applique les six règles de validité. En cas d'échec, elle renvoie `ErreurLog::FormatInvalide` avec la ligne **sans ses espaces autour**.

Démarche conseillée :

1. retirer les espaces autour de la ligne ;
2. découper en **au plus 4 morceaux** sur les espaces : `ligne.splitn(4, ' ')` (le 4e morceau contient tout le reste, espaces compris) ; collecter dans un `Vec<&str>` et vérifier qu'il y en a bien 4 ;
3. vérifier la date et l'heure avec la fonction utilitaire ci-dessous ;
4. vérifier les crochets (`starts_with`, `ends_with`), extraire le texte entre les deux avec une tranche `&crochets[1..crochets.len() - 1]`, puis le convertir avec `Niveau::depuis_texte` ;
5. séparer le module du message avec `reste.split_once(": ")`, qui renvoie une `Option<(&str, &str)>` ;
6. construire l'`EntreeLog`.

Fonction utilitaire fournie, à recopier (privée, dans `entree.rs`) :

```

/// Vrai si `texte` a exactement la forme du modèle, où « 9 » représente
/// un chiffre et tout autre caractère doit être présent tel quel.
/// Exemple : respecte_modele("2026-10-06", "9999-99-99") vaut true.
fn respecte_modele(texte: &str, modele: &str) -> bool {
    if texte.len() != modele.len() {
        return false;
    }
    for (c, m) in texte.chars().zip(modele.chars()) {
        let ok = if m == '9' { c.is_ascii_digit() } else { c == m };
        if !ok {
            return false;
        }
    }
    true
}

```

Astuce : de nombreux contrôles renvoient la même erreur. Construisez-la une fois au début (`let invalide = ErreurLog::FormatInvalide(ligne.to_string());`), puis écrivez `return Err(invalide);` à chaque contrôle. Pourquoi peut-on « réutiliser » `invalide` à plusieurs endroits alors qu’il n’est pas `Copy` ? (Indice : combien de ces `return` s’exécutent au plus ?)

Test : `cargo test --test palier2_entree` (9 tests).

Palier 3 — Chargement d’un journal (25 min)

Dans `chargement.rs` :

```

pub fn charger_texte(contenu: &str) -> (Vec<EntreeLog>, Vec<ErreurLog>)
pub fn charger_fichier(chemin: &str)
    -> Result<(Vec<EntreeLog>, Vec<ErreurLog>), ErreurLog>

```

- `charger_texte` parcourt les lignes (`contenu.lines()`), ignore les lignes vides et les commentaires `#`, et range chaque ligne dans les entrées valides ou dans les erreurs, **dans l’ordre du fichier**. Elle ne peut pas échouer.
- `charger_fichier` lit le fichier avec `std::fs::read_to_string`, convertit l’éventuelle erreur d’entrée/sortie en `ErreurLog::Fichier(chemin)` avec `map_err`, puis délègue à `charger_texte`. Utilisez l’opérateur `?`.

Séparer ainsi la lecture du fichier de son analyse permet de tester l’analyse sans fichier.

Test : `cargo test --test palier3_chargement` (4 tests). Le fichier fourni doit donner **27 entrées valides et 4 lignes invalides**. Repérez-les dans `logs/serveur.log` : pourquoi chacune est-elle invalide ?

Palier 4 — Statistiques, filtres et rapport (50 min)

Dans `analyse.rs`, écrivez les fonctions suivantes. Les filtres renvoient des **copies** des entrées sélectionnées (`e.clone()`).

Fonction	Comportement
<code>compter_par_niveau(entrees: &[EntreeLog]) -> [usize; 4]</code>	Nombre d’entrées par niveau, dans l’ordre DEBUG, INFO, WARN, ERROR (utilisez <code>indice()</code>)
<code>filtrer_par_niveau(entrees: &[EntreeLog], niveau_min: Niveau) -> Vec<EntreeLog></code>	Entrées de niveau supérieur ou égal à <code>niveau_min</code>
<code>filtrer_par_module(entrees: &[EntreeLog], module: &str) -> Vec<EntreeLog></code>	Entrées du module, sans tenir compte des majuscules ni des espaces autour du texte cherché
<code>rechercher(entrees: &[EntreeLog], mot: &str) -> Vec<EntreeLog></code>	Entrées dont le message contient le mot, sans tenir compte des majuscules ; un mot vide (ou fait d’espaces) ne donne aucun résultat
<code>dernieres_erreurs(entrees: &[EntreeLog], n: usize) -> Vec<EntreeLog></code>	Les <code>n</code> dernières entrées ERROR, dans l’ordre du fichier ; s’il y en a moins de <code>n</code> , toutes
<code>compter_par_module(entrees: &[EntreeLog]) -> Vec<(String, usize)></code>	Nombre d’entrées par module (nom exact, majuscules comprises), trié par nombre décroissant , puis par nom en cas d’égalité
<code>generer_rapport(entrees: &[EntreeLog], nb_lignes_invalides: usize) -> String</code>	Le rapport complet, au format ci-dessous

Indices

- `dernieres_erreurs` : récupérez d'abord toutes les erreurs, puis calculez l'indice de départ. Attention, `erreurs.len() - n` panique si `n` est plus grand (dépassement d'entier) : utilisez `saturating_sub`.
- `compter_par_module` : sans `HashMap`, utilisez un `Vec<(String, usize)>`. Pour chaque entrée, cherchez le module dans le vecteur (`iter_mut()`) ; s'il est absent, ajoutez-le avec un compteur à 1. Pour le tri, la méthode `sort_by` reçoit une fonction de comparaison (une *fermeture*) ; recopiez-la telle quelle :

```
compteurs.sort_by(|a, b| b.1.cmp(&a.1).then(a.0.cmp(&b.0)));
```

- `generer_rapport` : construisez une `String` vide, puis ajoutez les lignes avec `push_str(&format!(...))`. La période va de la date et l'heure de la **première** entrée à celles de la **dernière** (`first()` et `last()` renvoient des `Option`). Les dernières erreurs sont les 5 dernières.

Format exact du rapport (exemple avec 7 entrées, dont 3 erreurs, et 2 lignes invalides ; chaque ligne se termine par un retour à la ligne, et les lignes de détail sont indentées de deux espaces) :

```
=== Rapport d'analyse ===
Entrées valides : 7
Lignes invalides : 2
Période : 2026-10-06 08:00:00 -> 2026-10-06 11:00:00

Répartition par niveau :
  DEBUG : 1
  INFO : 2
  WARN : 1
  ERROR : 3

Modules :
  auth : 2
  Auth : 1
  api : 1
  bdd : 1
  cache : 1
  serveur : 1

Dernières erreurs :
  2026-10-06 08:11:00 [ERROR] auth: compte bloqué
  2026-10-06 09:30:00 [ERROR] bdd: délai dépassé
  2026-10-06 11:00:00 [ERROR] api: erreur 500
```

Sans aucune entrée, la ligne de période devient `Période : aucune entrée`, et la section des erreurs contient une seule ligne `aucune`.

Test : `cargo test --test palier4_analyse` (8 tests). Une fois les quatre paliers terminés, `cargo test` lance les 27 tests d'un coup.

Palier 5 — Le menu interactif (45 min)

Dans `main.rs`, écrivez le programme principal.

Comportement attendu

Au démarrage, le programme charge le fichier passé en argument (`cargo run -- logs/autre.log`) ou, à défaut, `logs/serveur.log` . Il affiche le résultat du chargement, ou l'erreur si le fichier est illisible ; dans ce cas, le programme continue **sans fichier chargé**.

Puis il affiche le menu en boucle :

```
===== Analyseur de logs =====
Fichier : logs/serveur.log (27 entrées)
1. Charger un fichier
2. Statistiques
3. Filtrer par niveau minimum
4. Filtrer par module
5. Rechercher un mot dans les messages
6. Dernières erreurs
7. Afficher les lignes invalides
8. Exporter le rapport dans un fichier
0. Quitter
Votre choix :
```

La deuxième ligne indique `Aucun fichier chargé` tant qu'aucun fichier n'a pu être lu.

Choix	Action
1	Demande un chemin et charge le fichier. En cas d'erreur, l'ancien journal reste chargé.
2	Affiche le rapport (<code>generer_rapport</code>)
3	Demande un niveau ; affiche les entrées filtrées, ou « Niveau inconnu »
4	Demande un module ; affiche les entrées filtrées
5	Demande un mot ; affiche les entrées trouvées
6	Demande un nombre (5 si la saisie est vide) ; affiche les dernières erreurs, ou « Nombre invalide »
7	Affiche les lignes invalides du dernier chargement
8	Demande un nom de fichier (<code>rapport.txt</code> si vide) et y écrit le rapport (<code>std::fs::write</code>)
0	Affiche « Au revoir ! » et quitte

- Les options 2 à 8 exigent un fichier chargé : sinon, afficher « Chargez d'abord un fichier (option 1). ».
- Les listes d'entrées se terminent par le nombre de résultats, ou affichent « Aucun résultat. ».
- Toute autre saisie affiche « Choix invalide » et réaffiche le menu.
- **Le programme ne doit jamais planter**, quelle que soit la saisie.

Conseils

Conserver l'état : regroupez le journal chargé dans une structure, et gardez-la dans une `Option`, qui vaut `None` tant qu'aucun fichier n'est chargé :

```
struct Session {
    chemin: String,
    entrees: Vec<EntreeLog>,
    erreurs: Vec<ErreurLog>,
}

let mut session: Option<Session> = /* chargement initial */;
```

Lire une saisie : écrivez une fonction utilitaire. `print!` n'affiche rien tant que le tampon n'est pas vidé, d'où le `flush()` :

```

use std::io::{self, Write};

fn demander(invite: &str) -> Option<String> {
    print!("{invite}");
    io::stdout().flush().expect("affichage impossible");
    let mut ligne = String::new();
    match io::stdin().read_line(&mut ligne) {
        Ok(0) | Err(_) => None, // fin de l'entrée : on quittera
        Ok(_) => Some(ligne.trim().to_string()),
    }
}

```

Structurer la boucle : un `loop` contenant un `match` sur `choix.as_str()`. Découpez en petites fonctions (`charger`, `afficher_menu`, `afficher`) plutôt que d'écrire un `main` de 150 lignes.

Recette : testez au minimum, à la main :

- chaque option avec `logs/serveur.log` ;
- un niveau inconnu (`FATAL`), un nombre invalide (`abc`), un choix invalide (`9`, lettre, saisie vide) ;
- le lancement avec un fichier absent (`cargo run -- absent.log`), puis l'option 2, puis le chargement de `logs/serveur.log` avec l'option 1 ;
- l'export, puis l'ouverture du fichier produit.

Avant de rendre : `cargo fmt`, puis `cargo clippy` ne doit afficher aucun avertissement.

Bonus

À traiter dans l'ordre de votre choix, après avoir terminé les paliers 0 à 5 :

1. **Filtre par plage horaire** : afficher les entrées entre deux heures saisies (`09:00:00` et `12:00:00`). Les heures au format `HH:MM:SS` se comparent directement comme des chaînes : pourquoi ?
2. **Combinaison de filtres** : niveau minimum et module en une seule option.
3. **Export des résultats** : après un filtre, proposer d'enregistrer les entrées affichées dans un fichier.
4. **Mode non interactif** : `cargo run -- logs/serveur.log --rapport` affiche le rapport et quitte sans menu (utile dans un script).
5. **Statistiques par heure** : nombre d'entrées pour chaque heure de la journée (`08h`, `09h`...), sous forme d'histogramme textuel (`09h █ 4`).
6. **Vos propres tests** : ajoutez dans `src/` des tests unitaires (`#[cfg(test)]`) pour la fonction `respekte_modele`, qui est privée et donc invisible depuis le dossier `tests/`.

Annexe — Aide-mémoire

Besoin	Code
Retirer les espaces autour	<code>texte.trim()</code>
Majuscules / minuscules	<code>texte.to_uppercase()</code> , <code>texte.to_lowercase()</code>
Comparer une <code>String</code> dans un <code>match</code>	<code>match s.as_str() { "a" => ..., _ => ... }</code>
Découper en au plus n morceaux	<code>ligne.splitn(4, ' ').collect::<Vec<&str>>()</code>
Couper au premier séparateur	<code>texte.split_once(": ") -> Option<(&str, &str)></code>
Commence / finit par	<code>s.starts_with([' '])</code> , <code>s.ends_with([' '])</code>
Contient	<code>message.contains(&mot)</code>
Tranche d'une chaîne	<code>&s[1..s.len() - 1]</code>
Parcourir les lignes	<code>for ligne in contenu.lines()</code>
Lire un fichier	<code>std::fs::read_to_string(chemin) -> Result<String, _></code>
Écrire un fichier	<code>std::fs::write(chemin, texte) -> Result<(), _></code>
Convertir une erreur	<code>resultat.map_err(_ MonErreur::...)?</code>
Arguments du programme	<code>std::env::args().collect::<Vec<String>>()</code>
Élément d'un <code>Vec</code> sans paniquer	<code>v.get(1) -> Option<&T></code>
Premier / dernier	<code>v.first()</code> , <code>v.last()</code> -> <code>Option<&T></code>
Soustraction sans dépassement	<code>a.saturating_sub(b)</code>
Nombre depuis une saisie	<code>texte.parse::<usize>() -> Result<usize, _></code>

